Es hora de que pongas en práctica todo lo aprendido. 🤓
Este apartado tiene el objetivo de ayudarte a seguir potenciando tus habilidades, por lo que a continuación encontrarás diferentes **desafíos** que podrás resolver de forma independiente y a tu ritmo.

Más adelante conseguirás las resoluciones para que valides tus respuestas y puedas monitorear tu progreso. 😉

¡Manos a la obra!

1. **Desafío** 🎯

   Construir, entrenar y evaluar un modelo de clasificación utilizando el algoritmo K-Nearest Neighbors (K-NN).

   Consigna

   Utilizando el siguiente conjunto de datos ficticio de personas, realiza los siguientes pasos:

	Altura (cm)	**Peso (kg)**
1	180	75
2	165	65
3	170	70
4	160	60

- *Aplica un* **escalamiento de datos** *utilizando Min-Max Scaling.*
- *Implementa un modelo de clasificación K-NN con* **k=3**.
- *Predice el deporte favorito de una persona nueva con altura de 172 cm y peso de 72 kg.*
- *Calcula e imprime la* **exactitud** *del modelo utilizando los datos*

originales.

- *Construye y muestra la* ***matriz de confusión****.*
- *Cambia el valor de* ***k*** *a 1 y 4 y analiza el efecto en la exactitud.*
- *Reflexiona: ¿Cómo afecta la selección del valor de k al desempeño del modelo?*

2. **¿Dónde se lleva a cabo?** 👩‍💻

   Visual Studio Code

3. **Tiempo de dedicación** ⏳

   2 horas

4. **Recursos**🛠️

   - https://scikit-learn.org/stable/modules/neighbors.html
   - https://machinelearningmastery.com/k-nearest-neighbors-for-machine-learning/

5. **Plus** ➕

   ✅ *Implementar la clasificación utilizando la* ***distancia Manhattan*** *y comparar los resultados.*

   ✅ *Aplicar una validación cruzada de 5 iteraciones (K-Fold) para evaluar el modelo.*

6. ⚠️**Condición**

   Esta práctica o ejercitación **no requiere ser entregada y/o evaluada** por el mentor. No obstante puedes compartir tus resultados con el resto de los bootcampers y construir conocimiento en conjunto.

Resolución del desafío 👇

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import MinMaxScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix

# Datos
data = {
    'Altura': [180, 165, 170, 160],
    'Peso': [75, 65, 70, 60],
    'Deporte': ['Fútbol', 'Baloncesto', 'Fútbol', 'Baloncesto']
}

df = pd.DataFrame(data)

# Variables independientes y dependiente
X = df[['Altura', 'Peso']]
y = df['Deporte']

# Escalamiento Min-Max
scaler = MinMaxScaler()
X_scaled = scaler.fit_transform(X)
```

```
# Modelo K-NN con k=3
knn = KNeighborsClassifier(n_neighbors=3)
knn.fit(X_scaled, y)

# Predicción nueva observación
nueva_persona = scaler.transform([[172, 72]])
prediccion = knn.predict(nueva_persona)
print(f"Predicción para la nueva persona: {prediccion[0]}")

# Predicciones y evaluación
y_pred = knn.predict(X_scaled)
accuracy = accuracy_score(y, y_pred)
cm = confusion_matrix(y, y_pred)

print(f"\nExactitud con k=3: {accuracy:.2f}")
print(f"Matriz de confusión:\n{cm}")

# Análisis para k=1 y k=4
for k in [1, 4]:
    knn_k = KNeighborsClassifier(n_neighbors=k)
    knn_k.fit(X_scaled, y)
    y_pred_k = knn_k.predict(X_scaled)
    accuracy_k = accuracy_score(y, y_pred_k)
    print(f"Exactitud con k={k}: {accuracy_k:.2f}")
```